

Lösungen Präsenzblatt 05

June 3, 2025

1 Lösungen des Präsenzblattes

Import der Primzahldatei

```
[1]: with open("primes.csv") as f:
      primes = []
      for line in f:
          for prime in line.split(", "):
              primes.append(int(prime))
```

Test: Funktioniert der Import?

```
[3]: primes[0:3]
```

```
[3]: [2, 3, 5]
```

1.1 Lösung Aufgabe 1

```
[8]: def nth_primzahlzwilling(n):
      anzahl = 0
      for prime in primes:
          if prime + 2 in primes:
              # prime und prime + 2 sind Primzahlzwillinge
              anzahl += 1
          if anzahl == n:
              return (prime, prime+2)
      return -1
```

```
[9]: nth_primzahlzwilling(7)
```

```
[9]: (59, 61)
```

1.2 Lösung Aufgabe 2

```
[6]: def anz_primzahlzwillinge(von, bis):
      prime_slice = [x for x in primes if x >= von and x <= bis]
      anzahl = 0
      for prime in prime_slice:
          if prime + 2 in prime_slice:
```

```

        # prime und prime + 2 sind Primzahlzwillinge
        # im angegebenen Bereich
        anzahl += 1
    return anzahl

```

```
[7]: anz_primzahlzwillinge(2000, 4000)
```

```
[7]: 42
```

1.3 Lösung Aufgabe 3

```
[10]: def append_char(l: list):
        new_list = []
        for item in l:
            for x in "ABCDEFGH":
                new_list.append(item + x)
        return new_list

```

```
[13]: l = ["A", "B", "C", "D", "E", "F", "G", "H"]
        l = append_char(l)
        l = append_char(l)
        l = append_char(l)

```

In der Liste l sind nun alle vierstelligen Strings aus den Buchstaben A bis H

```
[16]: l[0:3]
```

```
[16]: ['AAAA', 'AAAB', 'AAAC']
```

Nun konstruieren wir eine neue Liste, in der nur noch Elemente sind, die mindestens eine der drei gegebenen Eigenschaften haben. Da wir auf Strings wir auf Listen arbeiten können, können wir auf das erste, zweite und dritte Element folgendermaßen zugreifen:

```
[17]: l = [x for x in l if x[0] != 'A' or x[1] == 'B' or x[2] == 'C']

```

Die Länge der neuen Liste ist nun die Anzahl an Strings, die eine der Eigenschaften haben

```
[18]: len(l)
```

```
[18]: 3704
```

1.4 Lösung Aufgabe 4

```
[31]: def dreieckszahlen(n):
        d = [1]
        for i in range(2, n+1):
            d.append(d[-1] + i)
        return d

```

```
[37]: dreieckszahlen(100)[0:10]
```

```
[37]: [1, 3, 6, 10, 15, 21, 28, 36, 45, 55]
```

```
[38]: def quadratzahlen(n):  
      q = [1]  
      for i in range(2, n+1):  
          q.append(i**2)  
      return q
```

```
[39]: quadratzahlen(100)[0:10]
```

```
[39]: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

```
[44]: l = [x for x in dreieckszahlen(500) if x in quadratzahlen(500)]
```

```
[45]: len(l)
```

```
[45]: 4
```

```
[46]: l
```

```
[46]: [1, 36, 1225, 41616]
```

1.5 Lösung Aufgabe 5

```
[47]: def fiblist(n):  
      if n == 1:  
          return [1]  
      else:  
          fibs = [1,1]  
          for i in range(n-2):  
              fibs.append(fibs[-1]+fibs[-2])  
          return fibs
```

Wir versuchen einfach mal, ob die gesuchte Zahl unter den ersten 500 Zahlen liegt. Wenn nein, müssen wir einen anderen Weg wählen

```
[48]: l = [x for x in fiblist(500) if x in quadratzahlen(500)]
```

```
[49]: l
```

```
[49]: [1, 1, 144]
```

Was gäbe es aber sonst für eine Alternative?

```
[52]: import math
```

```
[56]: def ist_quadratzahl(n):  
      return math.sqrt(n) == int(math.sqrt(n))  
  
      def ist_quadratzahl(n):  
          return n**(1/2) == int(n ** (1/2))
```

```
[58]: ist_quadratzahl(144)
```

```
[58]: True
```